

The Cloud Migration Playbook

A practical framework for small teams moving infrastructure to a simpler, cheaper, more observable cloud setup, without a big-bang rewrite.

By [Themistoklis Baltzakis](#), [cloudless.gr](#)

How to use this playbook

This playbook is written for small teams (roughly 2 to 20 people) that are past the MVP stage: the product works, but the infrastructure grew by accident, the cloud bill is hard to explain, and nobody owns operations full-time.

Work through the steps in order. Each step ends with a short checklist. Nothing here requires a specific vendor; where we mention Cloudflare, GCP or Azure it is because those are the platforms we build on with clients.

A note on numbers: this playbook deliberately contains no benchmark figures or "average savings" claims. The only numbers that matter are the ones you measure on your own system in Step 2, and compare against after the move.

Step 0: Decide whether you should migrate at all

Migration is a cost, not a goal. Good reasons to migrate:

- You cannot explain what a large share of your monthly bill is for.
- You pay for capacity that sits idle most of the day.
- Deploys are manual, risky, or depend on one person.
- You have had outages you could not diagnose because there were no logs or metrics.
- A single server or region is a single point of failure for the whole business.

Weak reasons to migrate:

- "Everyone is moving to serverless."
- A new platform has a generous free tier, but your current setup is stable and cheap.
- You want to rewrite the application anyway. Do the rewrite or the migration, not both at once.

Checklist:

- ☐ Write down, in one sentence each, the two or three problems the migration must solve.
- ☐ Agree on what "done" looks like before any work starts.

Step 1: Inventory everything that runs

You cannot migrate what you have not listed. Build one table, one row per component:

- Compute: servers, containers, functions, cron jobs, background workers, queues.
- Data: databases, caches, object storage buckets, file shares, backups.
- Network: domains, DNS zones, load balancers, CDNs, TLS certificates, firewalls, VPNs.
- Identity and secrets: API keys, service accounts, OAuth apps, where each secret is stored.
- Third parties: email, payments, analytics, monitoring, anything with a webhook pointing at you.

For each row record: owner, what depends on it, what it depends on, monthly cost line, and whether it holds state.

Checklist:

- ☐ Every line on the current cloud bill maps to a row in the inventory.
- ☐ Every row has a named owner.
- ☐ Stateful components are clearly marked; they are migrated last.

Step 2: Measure a baseline before you move

Record the numbers you will use to judge the migration. Measure them on your system, over a normal week:

- Monthly cost, broken down by service and environment.
- Request latency (p50 and p95) for your most important endpoints.
- Error rate for those endpoints.
- How long a deploy takes, and how often you deploy.
- How long it takes to recover from a bad deploy.
- Data volumes: database size, object storage size, monthly egress.

If you cannot measure one of these today, that is your first finding. Add basic logging and metrics before migrating, so you are not comparing against guesses.

Checklist:

- ☐ Baseline numbers are written down with the date and how they were measured.
- ☐ You have at least logs and request metrics for the critical path.

Step 3: Classify each workload

For every compute and data row in the inventory, pick exactly one path:

- Retire: nobody uses it. Turn it off (after checking logs), keep a backup.
- Retain: it works, it is cheap, it is not part of the problem. Leave it.
- Rehost: move it as-is (same container or VM) to the new platform. Fastest, least benefit.
- Replatform: small changes to use a managed service (for example a managed database or object storage instead of a self-run one).
- Refactor: rebuild it for the target platform, for example as serverless functions. Highest effort, only where the payoff is clear.

Signs a workload is a good serverless fit:

- Request/response or event-driven, stateless between requests.
- Traffic is bursty or idle much of the time.
- Short execution per request.

Signs it is a poor fit (keep it on containers or VMs):

- Long-running CPU or GPU jobs.
- Heavy in-memory state or long-lived connections that the platform does not support well.
- Dependencies on native binaries or OS features that the runtime does not provide.

Checklist:

- ☐ Every row has one of: retire, retain, rehost, replatform, refactor.
- ☐ Refactor is chosen only where you can state the concrete benefit.

Step 4: Choose the target architecture

Pick the simplest architecture that solves the problems from Step 0. A typical pattern for small teams on Cloudflare:

- DNS, TLS and CDN at the edge.
- Stateless APIs as Workers.
- Relational data in D1 (or a managed Postgres where you need full Postgres features).
- Files and media in R2 object storage, which does not charge egress fees.
- Queues or scheduled triggers for background work.

On GCP or Azure the same shape applies with their equivalents (managed containers or functions, a managed database, object storage, a managed queue). What matters is the shape: stateless compute, managed state, and nothing that needs a person to keep it alive.

Decide explicitly:

- Where each secret will live and who can read it.
- How environments are separated (at minimum: production and one non-production environment).
- How you will deploy (from CI, never from a laptop).

Checklist:

- ☐ A one-page diagram of the target: compute, data, network, secrets.
- ☐ Every inventory row has a destination on the diagram (or is retired).

Step 5: Plan the migration order and cutover

Move in small, reversible steps. A safe order:

- 1 DNS and edge first (so you can route traffic gradually later).
- 2 Static assets and object storage.
- 3 Stateless services, one at a time.
- 4 Background jobs and schedulers.
- 5 Databases and other state, last.

Cutover patterns:

- Strangler pattern: route one path or endpoint at a time to the new system, keep the rest on the old one.
- Blue/green: run old and new side by side, switch traffic at the load balancer or DNS, switch back if needed.
- Gradual rollout: send a small share of traffic to the new system first, then increase.

For data:

- Prefer snapshot plus replay of changes, or a short write freeze, over "copy and hope".
- Verify row counts and checksums on the critical tables after copying.
- Keep the old database read-only (not deleted) until the new one has run cleanly for a while.

Every step needs a written rollback: what you switch back, who does it, and how long it takes.

Checklist:

- ☐ Each migration step fits in a single working session.
- ☐ Each step has a tested rollback.
- ☐ Nothing stateful is deleted until after the post-migration review.

Step 6: Put cost guardrails in place

A migration can make costs worse if nobody watches them. From day one:

- Set budget alerts on the new account or project.
- Tag or label every resource with owner and environment.
- Watch data transfer (egress) between regions, providers and the internet.
- Schedule or scale non-production environments down when unused.
- Set lifecycle rules on storage and log retention.
- Review the bill line by line after the first full month.

Checklist:

- ☐ Budget alerts go to a person who will act on them.
- ☐ Every resource is tagged with an owner.

Step 7: Security and operations

- Secrets live in the platform's secret store or a vault, never in the repository or in plain environment files that get copied around.
- Least privilege: separate credentials per service, scoped to what it needs.
- Backups are only real once a restore has been tested. Test one.
- Observability: logs, request metrics and error alerts for every service on the critical path.
- A short runbook per service: how to deploy, how to roll back, where the logs are, who to call.

Checklist:

- ☐ A restore from backup has been performed and timed.
- ☐ An alert fires when the critical path returns errors.
- ☐ Runbooks exist and are stored where the team can find them.

Step 8: Cutover day and post-migration review

Cutover day:

- ☐ Announce a window; freeze unrelated deploys.
- ☐ Lower DNS TTLs in advance if you will switch via DNS.
- ☐ Run the migration steps in order, checking health after each one.
- ☐ Keep the rollback owner online until traffic has been stable for a while.

Post-migration review, once the new setup has run for a normal week:

- Re-measure every baseline number from Step 2 with the same method.
- Compare, and write down what improved, what got worse, and why.
- Decommission the old resources only after this review, and only after a final backup.

Common mistakes

- Migrating and rewriting at the same time.
- Moving the database first.
- No baseline, so nobody can say whether the migration worked.
- Deleting the old system before the new one has proven itself.
- Keeping secrets in the repository "temporarily".
- No owner for the cloud bill after the move.

Next steps

If you want a second pair of eyes on your setup, book a free 30-minute audit. We review your infrastructure and give you a concrete action plan, with no commitment required:

<https://cloudless.gr/contact>

If you would rather build it yourself, the Serverless Masterclass walks through the Workers, D1 and R2 framework we run in production, with source code included:

<https://cloudless.gr/en/store/dig-serverless-course>

If you want to share your own migration results as you go, SocialAuto schedules and publishes posts across LinkedIn, X, Facebook, Instagram, TikTok and Threads from one place:

<https://social.cloudless.gr/pricing>

Appendix A: Inventory worksheet

Component	Type	Owner	Depends on	Holds state?	Monthly cost	Path

Appendix B: Baseline worksheet

Metric	How measured	Before	After
Monthly cost			
p50 latency (critical endpoint)			
p95 latency (critical endpoint)			
Error rate			
Deploy duration			
Time to recover from a bad deploy			